Cabled Robotic Arm Trajectory Optimization

1st Joshua Lee
Stanford University
Palo Alto, California
joshhlee@stanford.edu

***Abstract*—This project aims to optimize the trajectory of a robotic arm using a system of springs and pulleys, with actuators placed at the base to minimize moving mass. Our "N+1" cabling scheme reduces cable requirements and allows modulation of joint stiffness by adjusting tendon tensions. We have built a testbed with one link (N=1) and two tendons, incorporating sensors for tendon tension, output force, and angles. The primary objective is to achieve precise position control of the output link using MATLAB simulations to optimize motor effort and arm position for both positional accuracy and safety.**

Key metrics include accurate positioning with minimal effort, safety, and compliance with external disturbances. Optimization will be achieved using direct methods, namely Hooke-Jeeves, leveraging a pre-developed simple controller to refine system inputs. By comparing MATLAB simulations with the physical testbed, we aim to identify model errors and improve the system's accuracy. This methodology will be extended to a 3 DoF system with four cables (N+1 configuration).

I. Introduction

The optimization of robotic arm trajectories is a crucial area of research with significant implications for various applications, including manufacturing, healthcare, and service industries. Traditional robotic systems typically place actuators directly at the joints, which increases the moving mass and limits the agility and efficiency of the system. This project addresses these challenges by developing an innovative robotic arm design that utilizes a system of springs and pulleys, with actuators positioned at the base of the arm to minimize moving mass.

Our approach employs an "N+1" cabling scheme, which significantly reduces the number of cables needed for an N degree-of-freedom (DoF) system from 2N to N+1. This scheme not only reduces cable complexity but also allows for the modulation of joint stiffness through the adjustment of tendon tensions. This is achieved by varying the homogeneous component of the cables' tensions, enabling dynamic control of the system's stiffness based on the mechanical behavior of the tendons rather than relying solely on the servo system's bandwidth.

To develop and test this concept, we have constructed a testbed with one link (N=1) coupled to two motors via separate tendons made from stiffening materials such as rubber. The testbed is equipped with sensors for measuring tendon tension, output force, and motor and output link angles. Given the non-linear and time-varying dynamics of this design, an adaptive control system is essential to ensure stability, optimize performance, and monitor system components as they age over time.

The overall goal of the system is to be able to control a 3dof system across a trajectory. The understanding is that with a state space formulation and many modern techniques such as the Linear Quadratic Regulator (LQR), Model Predictive Control (MPC), and other techniques, we would be able to derive an optimal control trajectory. In this project, we explore whether a similar control cost is achievable using more simple techniques namely PD, if we use modern techniques to optimize and tune the gains.

The primary objective of this project is to achieve position control of the robotic arm. Position control involves reaching a commanded position initially using simpler linear springs and cables in this compliant design.

The value of using such a technique may be the simplicity in implementing more abstract cost metrics, which could be difficult to encode in LQR. Key optimization metrics include minimizing motor effort while achieving accurate positioning, preventing overshoot, ensuring system safety, and maintaining compliance with external disturbances. While these characteristics are desirable by any user, they are difficult to quantify as an output of states or inputs. Optimization will be pursued using Hooke-Jeeves, leveraging a pre-developed controller to refine system inputs. MATLAB simulations will play a crucial role in identifying model errors by comparing the simulated system to the physical testbed, thereby enhancing the model's accuracy. This process will also facilitate the extension of our approach to a 3 DoF system with four cables (N+1 configuration). The 3dof system will be simulated in matlab with the controller implemented once the system is built. We would hope that an optimized PD controller will perform similarly by some metrics and better by other metrics overall.

This report is organized as follows: Section II reintroduces the problem and scope of what we aim to explore, Section III provides background information. Section III details the methodology and experimental setup. Section IV presents the results and analysis. Section V discusses the implications and limitations of the findings.

II. Problem Setup

The goal in each simulation will be to track some reference trajectory. However, we will implement more and more system and safety constraints within the system. This includes the basic system requirements of keeping the cables under a certain amount of tension, minimizing position and velocity errors, or away from singularities. The safety side would be not exceeding specific tensions within the cables in case of

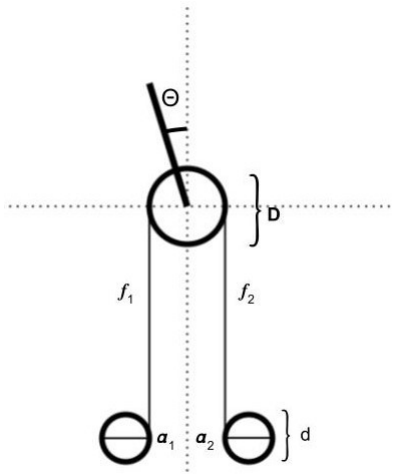

Fig. 1. Diagram of the Proposed system in 1dof

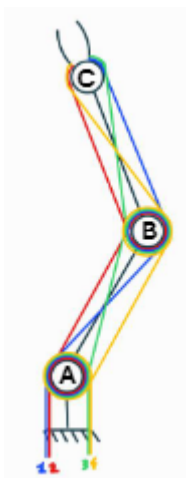

Fig. 2. Digram of 3dof System Configuration

backlash, remaining within certain velocity bounds even with large deflections, and lastly, random disturbances that impede the motion of the link enforcing safety. This performance will be compared with the cost of using a simple LQR controller that uses modern controls to track trajectories. This will then be implemented on the physical testbed and these various metrics will be measured to track model errors. Currently, there is a 1dof system 1 that is built. Using a similar technique, we would have a viable control paradigm to expand to the 3dof system. Using the 1dof case, we will have a relatively better idea of how this controller will perform on the 3dof system when we implement a similar one. For all of the matlab simulations, 3 and 4 generated trajectories will be used.

A. Control Scheme

For the classical control techniques, a simple PD controller was implemented and for the modern techniques to use as a baseline, an LQR controller was created. The cables that connect to the motors act as torque sources which actuate a

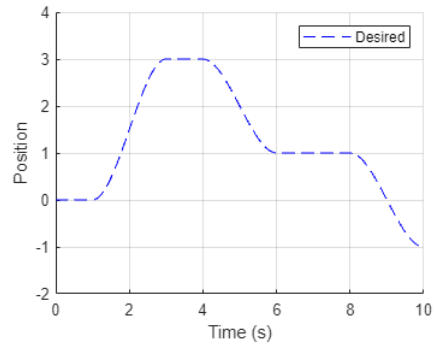

Fig. 3. Desired Simulated Positions

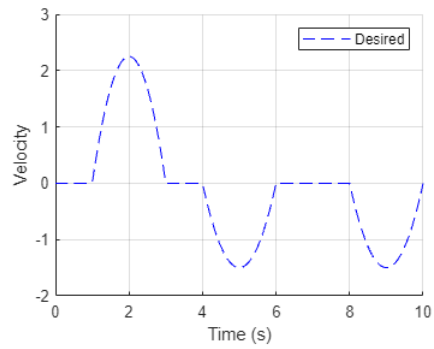

Fig. 4. Desired Simulated Velocities

set of springs that connect to the joint. This is from where the intrinsically compliant nature of the system arises. First, the PD controller, since the motors themselves are torque sources, they would ideally track the behavior of the springs, only exerting a torque or force specified by the controller irrespective of the dynamics of the springs. In other words, the springs will apply a variable force on the joint as they are stretched or compressed; however, we could remove this effect with the torque control since that specifies the force applied to the joint.

0: $force \leftarrow (-torque_1 + torque_2)$
0: $velocity \leftarrow velocity + force \times dt$
0: $position \leftarrow position + velocity \times dt$
0: $error \leftarrow desired_state - \begin{bmatrix} position \\ velocity \end{bmatrix}$
0: $torque_1 \leftarrow torque_1 - position_error \times Kp - velocity_error \times Kd$
0: $torque_2 \leftarrow torque_2 + position_error \times Kp + velocity_error \times Kd$ =0

For the state space LQR, the system can be simulated as a spring mass system with linear springs. This is the case since this is essentially two springs in series separated by a mass. Therefore, this can be reduced to just a simple spring mass system where we are controlling the position of the springs.

The state space problem builds as follows:

$$x = \begin{bmatrix} \theta \\ \dot{\theta} \end{bmatrix} \quad u = \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix}$$

$$A = \begin{bmatrix} 0 & 1 \\ \frac{-2k}{I} & 0 \end{bmatrix} \quad B = \begin{bmatrix} 0 & 0 \\ \frac{1}{I} & \frac{1}{I} \end{bmatrix}$$

$$C = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad D = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

B. Optimization

In the progression of the optimal control design, each additional constraint or metric as another step to be added into the overall cost or utility. First is the unconstrained optimization case, next is system requirements, and lastly safety constraints. The PD gains were optimized using Hooke-Jeeves algorithm. Hooke-Jeeves incrementally steps in an orthonormal basis of the variables to greedily converge to some local minimum. For the case of our PD controller, the two basis variables are the position gains and the derivative gains. This algorithm is as follows:

```
function HOOKE_JEEVES(f, x, α, ϵ, γ = 0.5)
    y ← f(x)
    n ← length(x)
    while α > ϵ do
        improved ← false
        x_best, y_best ← x, y
        for i ← 1 to n do
            for sgn ∈ {−1, 1} do
                x′ ← x + sgn × α × basis(i, n)
                y′ ← f(x′)
                if y′ < y_best then
                    x_best, y_best, improved ← x′, y′, true
                end if
            end for
        end for
        x, y ← x_best, y_best
        if ¬improved then
            α ← α × γ
        end if
    end while
    return x
end function=0
```

We would like to optimize the behavior according to three different steps.

a) Simple trajectory tracking, unconstrained. As far as the simple case of trajectory tracking goes, the optimization metrics are an arbitrary scale of position and derivative gains for the PD controller and an arbitrary scale of weights for state costs (Q) and input cost (R). The function of cost for this simple case is just the squared-sum of state errors and input cost, much like the LQR case.

b) Additional cost when the cables go slack. This is when the motor's torque has a positive sign. This intuitively means the cable is attempting to compress the spring, which although fine mathematically, physically is impossible. For this case, the system will incur a quadratic penalty as the slackness criteria is approached. However, as the cable is slack, the state will incur both a quadratic as well as constant penalty.

c) Safety criteria. This is a bit arbitrary, since ideally, a planning algorithm would prevent an unsafe trajectory. We can imagine some unsafe behaviors being overcompensation for error in position and velocity leading to overshoot or inaccuracies, high tension, or any of these when combined with uncertainties. This will also be an additional set of penalties that are placed on the system. However, safety can aptly be described as robust to disturbances. Therefore, we can observe the behavior as we introduce disturbances to the system.

III. Results

A. 1 dof Matlab Simulations

As a baseline for the tests, we use the simple LQR Controller, using the same costs or penalties that will be used on the PD Controller

$$Q = \begin{bmatrix} 20 & 0 \\ 0 & 10 \end{bmatrix} \quad R = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

With these baseline simulations [5][6], we found the optimal

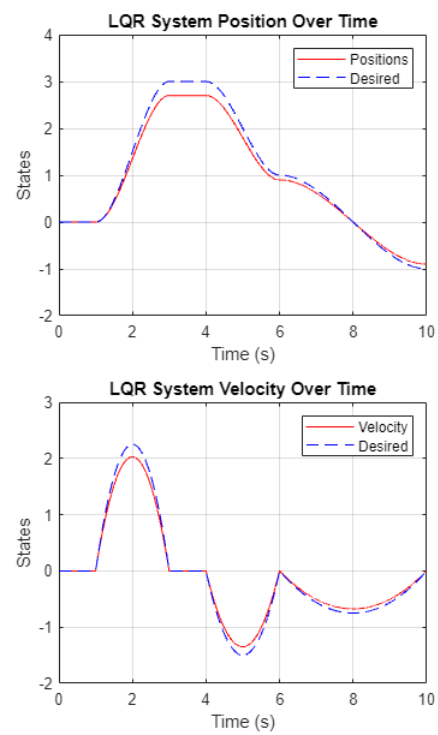

Fig. 5. Baseline LQR Control States

gain matrix to be $\begin{bmatrix} 45.25 & 0.7364 \\ 45.25 & 0.7364 \end{bmatrix}$. The final penalty from the LQR simulation from the state error and the input cost came out to 407.39. The errors were very stable and so the overall motion of the link from the LQR is smooth.

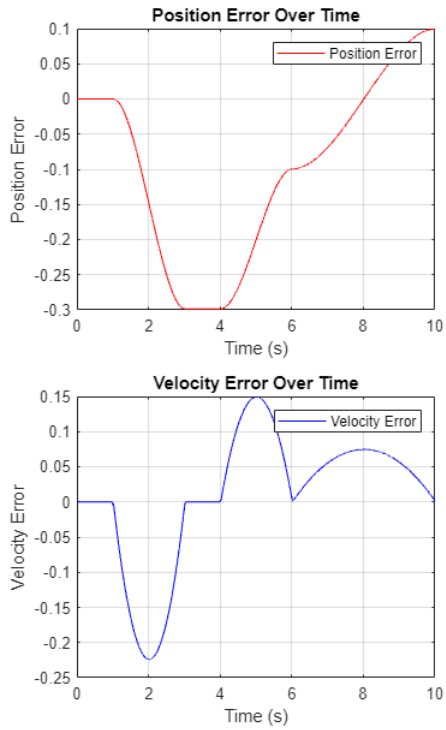

Fig. 6. Baseline LQR Control Errors

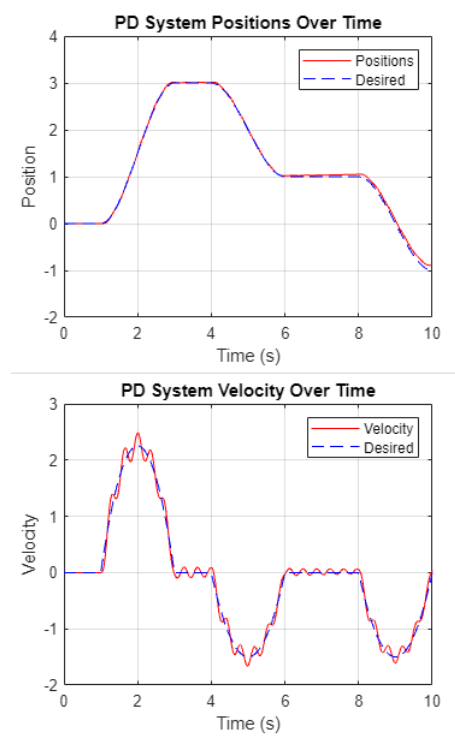

Fig. 7. States of tuned PD controller, Unconstrained case

1) Case (a): From test case a, we simply performed unconstrained optimization[7][8][9]. Using the hooke-jeeves algorithm, at an initial gains of (0,0), the gains converged to Kp = -1.039, Kd = 3.583. The cost of this control state error and

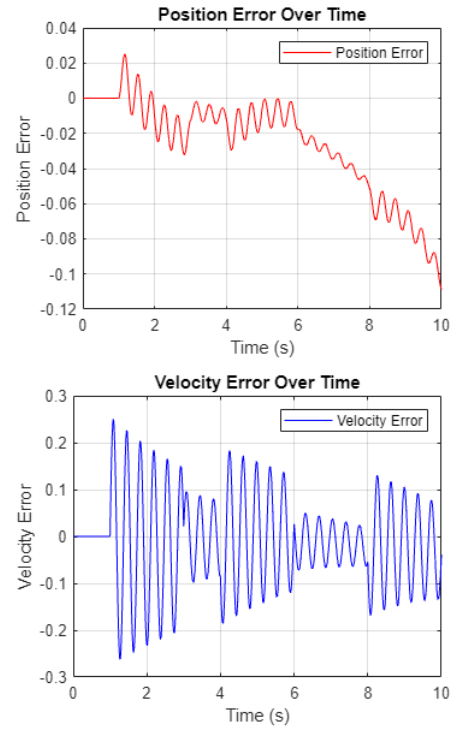

Fig. 8. Errors of PD controller, Unconstrained case

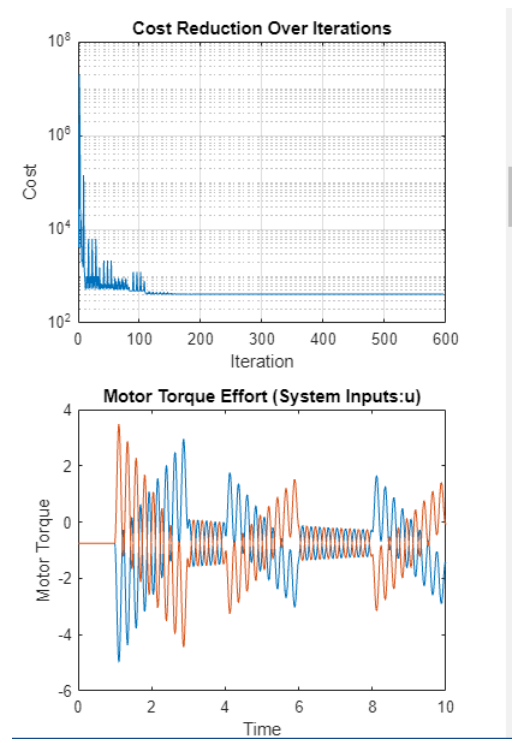

Fig. 9. Descent of total cost per iteration plotted on a Log scale and the motor torque output

and input effort was 409.6826. This was very good because it shows that a well tuned PD controller performed just as well

as a LQR controller. However, even though the overall notion of cost was the same for both the baseline case and the tuned PD controller, the LQR error was much smoother while the PD had lots of erratic motion and motion.

2) Case (b): For this case, we wanted to add system constraints that would force the system to match the true physical constraints[10][11][12]. Mainly that cables cannot apply a compressive force onto the springs. From figure 9, the motor torques quite frequently become positive even though this is not possible. To enforce the constraints, we will add a hybrid penalty at each state of the following form.

$$p(x) = \begin{cases} 0, & \text{if } torque_i < -1 \\ 100 \times (torque_i + 1)^2, & \text{if } -1 \leq torque_i < 0 \\ 100 \times (torque_i + 1)^2 + 5000, & \text{if } torque_i \geq 0 \end{cases} \tag{1}$$

This is just to ensure that with the gains provided, the motors would always continue to provide a tension through the cables. In addition, I added that if torques are greater than 0, they are deemed useless and therefore will provide a force of 0. Lastly, to scale properly, I had to add another gain term to scale with velocity, essentially such that the system would be more taut or loose depending on the velocity of the output joint. This particular function liked to have a higher tension during high velocities in order to make sure it kept its positional and velocity accuracies high. The result for the PD controller showed higher costs, but as expected, the overall motor torque effort remained before 0 for most of the duration of the control. Overall, the gains remained mostly similar, Kp = -1.33, Kd = 3.72, and Kv = 0.003, Kv being the velocity to tautness scaling term.

3) Case (c): For the safety, this one was a bit more nuanced to implement. The additional costs and penalty was relatively simple, incorporating safety was more difficult. What I did is added penalties to exceeding a certain velocity as well as penalties to exceeding a certain torque. In addition to the previous penalties, I added

$$p(x) = \begin{cases} 100 \times (torque_i + 5)^2, & \text{if } torque_i < -5 \\ 10 * (|velocity| - 5), & \text{if} velocity > 2.5 \end{cases} \tag{2}$$

However, the purpose of safety is to ensure that the system is robust to disturbances. In the case of a robotic arm system that is meant to be compliant, this could be an obstruction or a force that is inhibiting motion. For this, at the peak velocity or at approximately 2seconds, the velocity would be 0, meaning the trajectory is continuously computed with accumulating error while the system is unable to move. With a controller we had previously, we would assume the system would over compensate and jerk the system, causing it to overshoot. We would expect that with penalizing high torques and high velocities, the controller will behave safer. We were able to see that this was the case. The position tracking no longer became as relevant since it needed to remain within the bounds of torques and velocity. Once the lag occured, the position did not ever recover to catch up with the desired position. The final gains the algorithm settled at was Kp = 0.0544, Kd = 1.233, and Kv = 0.0137. What was extremely interesting to see was that once we implemented these safety constraints, the system naturally wanted to make the system

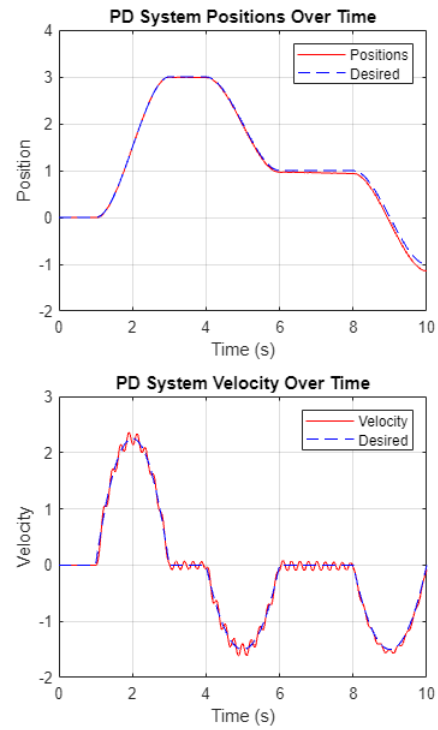

Fig. 10. States of PD controller, system constraints

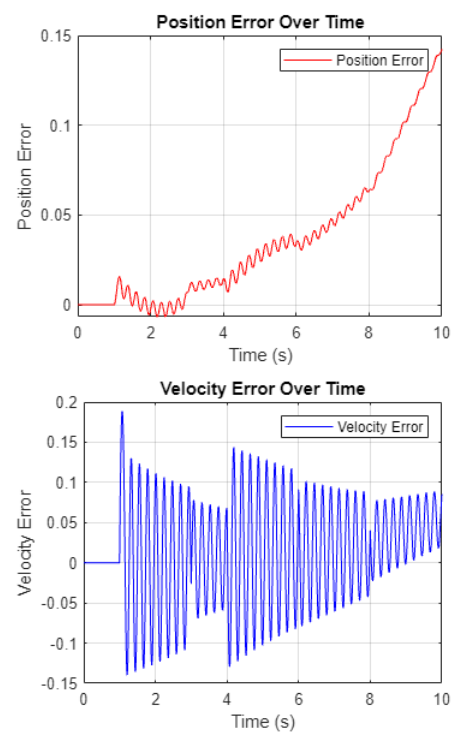

Fig. 11. Errors of PD controller, system constraints

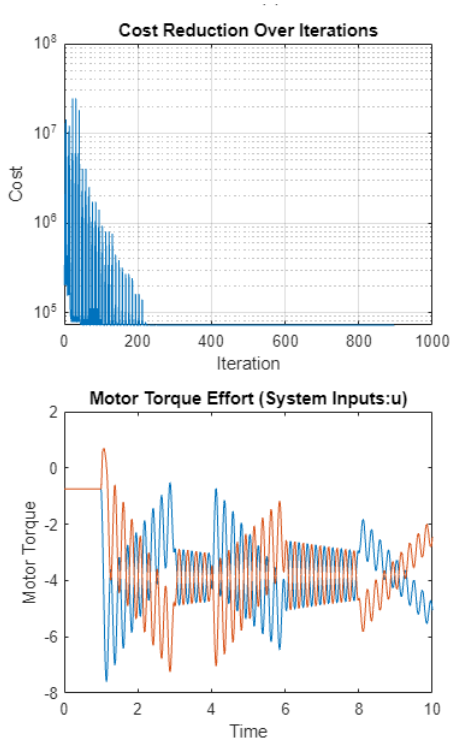

Fig. 12. Descent of total cost per iteration and the motor torque output, system constraints

more compliant as the velocity increased instead of tightening the cables to make the system more tight. The system preferred to lose the position tracking in order to stay within the bounds.

B. Testbed Experiments

In this section, we used the gains that we ended with at the final step implemented on the testbed and analyze the behavior under multiple scenarios[16][17]. Because of modeling errors, the exact gains that were used in the problem were not control gains. To make them work, I had to scale them to a point such that they did work on the system. The final gains I used on the system were Kp = 0.544, Kd = 12.33, and Kv = 0.137. These gains were completely incorrect however as the positions never quite reached the desired positions and when disturbed would go completely unstable. In order to come up with at least a viable solution, I resorted to handtuning the PD gains to at least gain some insight on what sort of results came out of the system. For position control, I found that gains of Kp=8, Kd=-0.5 worked relatively well.

C. 3dof Simulations/Future Work

The 3dof simulations were an extension of the single dof system, extending the assumption of the coupled spring in series simplification. The arm is currently being built using the same cabled mechanism and configuration from before. LQR controls perform very well for this, depending on the choice of penalties and costs. Building from the previous results from

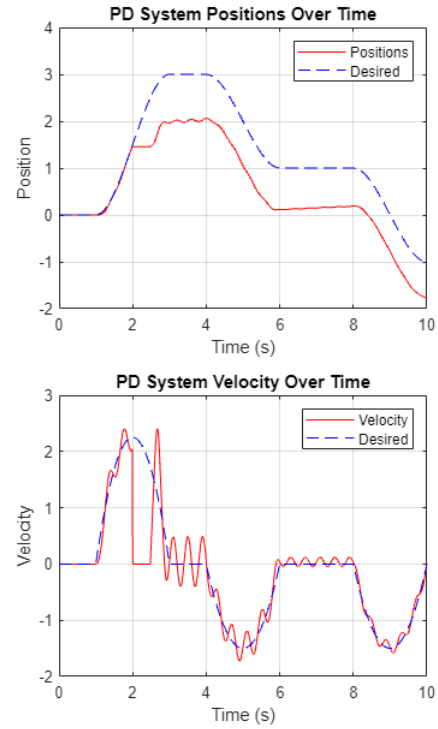

Fig. 13. States of PD controller, safety constraints

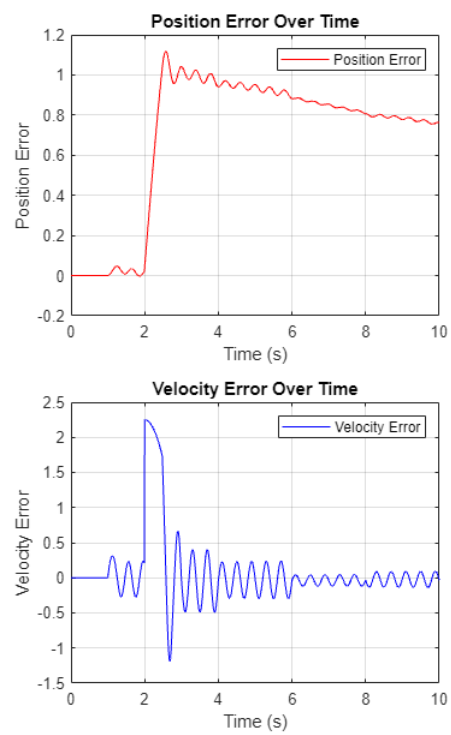

Fig. 14. Error of PD controller, safety constraints

the one dof system, it may be best to combine the initial LQR Gains and just slightly tune then as we introduce additional constraints, similar to the PD controller. Therefore, I used a

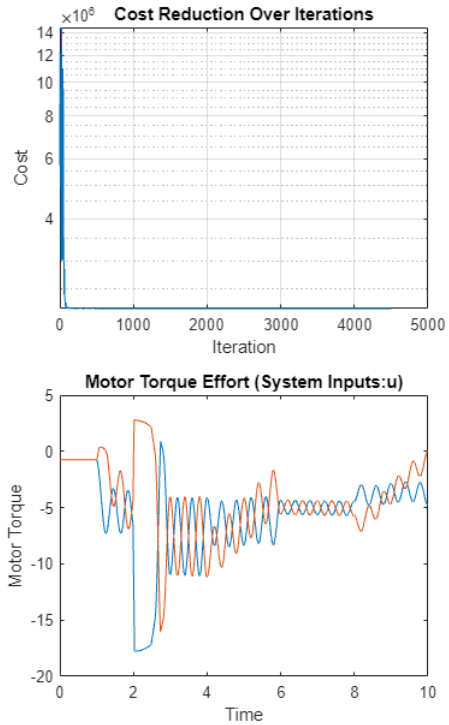

Fig. 15. Descent of total cost per iteration and the motor torque output, safety constraints

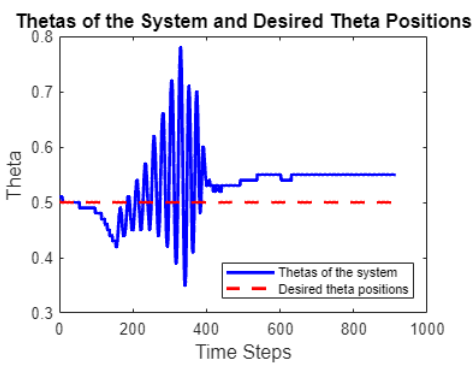

Fig. 16. Unstable and poor tracking of the gains found from before

state space formulation to control the system.

$$x = \begin{bmatrix} \theta_1 \\ \dot{\theta}_1 \\ \theta_2 \\ \dot{\theta}_2 \\ \theta_3 \\ \dot{\theta}_3 \end{bmatrix} \quad u = \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ \alpha_4 \end{bmatrix}$$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ -\frac{k}{m} & -w & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{k}{m} & -w & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & -\frac{k}{m} & -w \end{bmatrix}$$

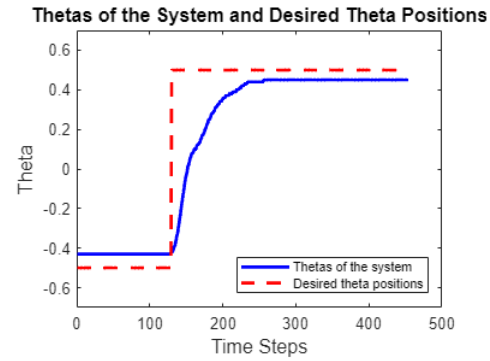

Fig. 17. Handtuned gains tracking the desired states

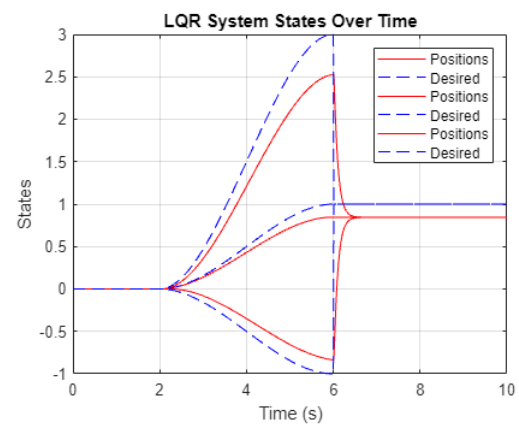

Fig. 18. LQR optimized trajectory for 3dof case

$$B = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & -1 & -1 \\ 0 & 0 & 0 & 0 \\ -1 & 1 & 1 & -1 \\ 0 & 0 & 0 & 0 \\ -1 & 1 & -1 & 1 \end{bmatrix}$$

With this formulation, we are able to compute the optimal trajectory in matlab simulations[18]. However, same as before, the control inputs are not actually feasible since cables go slack as they lose agency over the system. Also, the large spike in the motor actuation[19] could mean that there is a singularity point reached by the arm, from which one motor loses complete control of a degree of freedom18. Incorporating these additional constraints is a future step that I believe can be done.

IV. Discussion

The results presented in the previous sections provide valuable insights into the performance of the proposed trajectory optimization approach for cabled robotic arms. However, several important considerations and avenues for future exploration emerge from these findings.

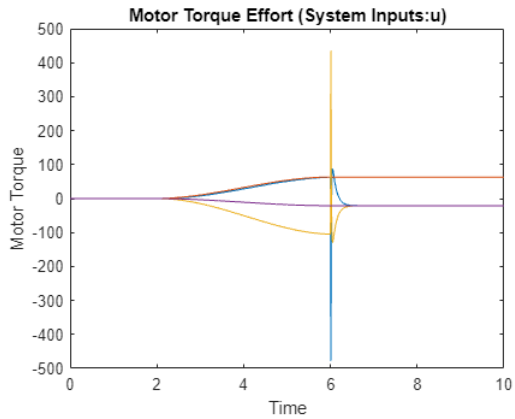

Fig. 19. LQR inputs for 3dof system

As far as the constrained optimization went, it did show lots of insight to how each constraint could change the behavior of the system, even when using simple direct methods. I could also see the limitations of the system. After the testbed integration behaved so poorly, I tried manually different choices of gains into the simulation. What I found that these direct methods was an extremely poor choice of finding the maximum because there were far too many local minimas where a methods such as hooke-jeeves could not escape from. For various step sizes, step size decreases, and learning rates, the solution changed drastically.

Seeing the PD gains tuned perform so poorly on the testbed was discouraging as the optimization in simulations were just that: simulations. They did not reflect the real world scenarios or physics. They were idealized models of what the system could behave like. In the end, the instability seemed to be caused by the incorrect assumptions that were made in the beginning that led to model mismatches and just unmodeled dynamics of the spring.

One notable observation is the disparity between the simulated results and the experimental outcomes obtained from the testbed. While the simulations demonstrated promising performance of the tuned PD controller, the testbed experiments revealed significant challenges in achieving stable and accurate trajectory tracking. This disconnect underscores the complexity of translating theoretical models into real-world applications and highlights the importance of accounting for practical considerations such as modeling errors and environmental factors.

Addressing Model Mismatches and Unmodeled Dynamics: The discrepancies observed between simulations and experiments emphasize the need for a more robust and accurate modeling framework. Future work could focus on refining the mathematical models used in simulations to better capture the dynamics of the cabled robotic arm system. Additionally, efforts to identify and incorporate unmodeled dynamics, such as friction, backlash, and nonlinearities, could improve the fidelity of simulations and enhance the predictive capability of control algorithms.

Integration of Safety Constraints: The incorporation of safety constraints in the trajectory optimization process represents a critical consideration for real-world applications of cabled robotic arms. While the simulations demonstrated the effectiveness of safety constraints in promoting stability and preventing unsafe behavior, their implementation on the testbed revealed practical challenges in ensuring compliance with safety criteria. Future work could focus on developing robust safety mechanisms and control strategies that prioritize system integrity and operator safety.

In conclusion, the findings presented in this study underscore the complex nature of trajectory optimization for cabled robotic arms. By addressing the challenges of model mismatches, exploring advanced control techniques, and integrating safety constraints, we can contribute to the development of robust and reliable control strategies for cabled robotic arm development.